

Intelligent Model Design

Last-Hour Formula and Symbol Sheet

Read this only after doing problems. It is designed for the final 20-30 minutes before the exam.

Numerical order: Given -> Formula -> Substitution -> Answer with shape/unit. State assumptions.

Universal symbols

Symbol	Meaning				
x, y, y_{hat}	input, true target, predicted target				
w, W, b	scalar/vector weight, weight matrix, bias				
θ	all trainable parameters				
L	loss; smaller is usually better				
E, M, B	epochs, sample count, batch size				
t	time-step index				
i, j	generic indices, such as class or feature				
$\exp(a)$	e raised to a				
$\ln(a)$	natural logarithm				
$\text{floor}(a)$	round down				
$\text{ceil}(a)$	round up				
$\text{sqrt}(a)$	square root				
T superscript	matrix transpose				
$\cdot, *$	element-wise multiplication				
$\cdot$		$\sum$		$\sum x $	sum of absolute values
$\cdot$		$\sum$		$\sum x^2$	sum of squared values

Training and metrics

```
batches_per_epoch = ceil(M/B)
total_updates = E * ceil(M/B)
```

M = number of training samples
 B = batch size
 E = epochs

```

accuracy = (TP+TN)/(TP+TN+FP+FN)
precision = TP/(TP+FP)
recall = TP/(TP+FN)
specificity = TN/(TN+FP)
F1 = 2*precision*recall/(precision+recall)

```

TP: true positive; TN: true negative; FP: false positive; FN: false negative.

Activation and loss

```

sigmoid(x) = 1/(1+exp(-x)) range (0,1)
tanh(x) range (-1,1)
ReLU(x) = max(0,x) range [0,infinity)
LeakyReLU(x) = max(alpha*x,x) alpha is small positive slope
softmax(z_i) = exp(z_i)/sum_j exp(z_j)
cross_entropy_for_true_class_c = -ln(p_c)

```

z_i : logit of class i ; p_c : probability of correct class c . Subtract maximum logit before softmax for numerical stability.

CNN shapes and parameters

```

K_eff = D(K-1)+1
N_out = floor((N_in+2P-D(K-1)-1)/S)+1

```

Symbol	Meaning
N_{in}, N_{out}	input and output height or width
K	physical kernel size
K_{eff}	effective size after dilation
D	dilation; ordinary convolution has $D=1$
P	zero padding on each side
S	stride
C_{in}, C_{out}	input channels and number of filters/output channels

Apply output formula separately to height and width. Output depth is C_{out} .

```

conv_params = (K_h*K_w*C_in+1)*C_out
dense_params = (N_in+1)*N_out

```

The +1 means bias. Remove it only if no bias. Pool, flatten, fixed upsampling, and ordinary activations have zero trainable parameters.

Receptive field

```

jump_l = jump_(l-1)*S_l
RF_l = RF_(l-1)+(K_eff_l-1)*jump_(l-1)
start: RF_0=1, jump_0=1

```

Three stride-1 3×3 convolutions: RF 3 $\rightarrow$ 5 $\rightarrow$ 7.

Old-paper anchors

```

20 input, K=5, S=1, P=0:
D=1 -> K_eff=5 -> 16 x 16
D=2 -> K_eff=9 -> 12 x 12
D=3 -> K_eff=13 -> 8 x 8

```

AlexNet Conv1: 227, K=11, S=4, 96 filters -> 55 x 55 x 96; parameters $(11*11*3+1)*96=34944$.

RNN

```

h_t = tanh(W_xh*x_t + W_hh*h_(t-1) + b_h)
y_t = softmax(W_hy*h_t + b_y)

```

x_t : input at time t ; h_t : hidden state; W_{xh} : input-hidden weights; W_{hh} : recurrent weights; W_{hy} : hidden-output weights.

```

simple_RNN_total = n_h(n_x+n_h+1) + n_y(n_h+1)

```

n_x, n_h, n_y : input, hidden, output dimensions. Do not multiply by sequence length; weights are shared.

LSTM

```

f_t = sigmoid(W_f[h_prev,x_t]+b_f)
i_t = sigmoid(W_i[h_prev,x_t]+b_i)
c_tilde_t = tanh(W_c[h_prev,x_t]+b_c)
c_t = f_t.*c_prev + i_t.*c_tilde_t
o_t = sigmoid(W_o[h_prev,x_t]+b_o)
h_t = o_t.*tanh(c_t)

```

```

LSTM_cell_params = 4*n_h*(n_x+n_h+1)

```

Symbol	Gate meaning
f_t	forget old cell memory
i_t	write candidate memory
c_{tilde_t}	candidate cell content
c_t	new long-term cell state
o_t	expose cell content
h_t	new hidden/output state

GRU - course convention

```

r_t = sigmoid(W_r[x_t,h_prev]+b_r)
h_tilde_t = tanh(W_h[x_t,r_t.*h_prev]+b_h)
z_t = sigmoid(W_z[x_t,h_prev]+b_z)
h_t = (1-z_t).*h_prev + z_t.*h_tilde_t

```

```

GRU_cell_params = 3*n_h*(n_x+n_h+1)

```

r_t : reset gate; z_t : update gate; high z_t favors candidate in this convention. State convention because some books reverse the mixing terms.

Attention and Transformer

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Object	Shape	Meaning
Q	$n_q \times d_k$	queries: what is sought
K	$n_k \times d_k$	keys: what is matched
V	$n_k \times d_v$	values: information retrieved
QK^T	$n_q \times n_k$	similarity scores
Output	$n_q \times d_v$	weighted values

Scale by $\sqrt{d_k}$ to keep softmax from saturating. Softmax each query row across keys.

```
d_head = d_model / number_of_heads
MHA = Concat(head_1, ..., head_h) W_O
```

Encoder cue: embedding+position -> MHA -> Add+Norm -> feed-forward -> Add+Norm. Pre-normalization is acceptable if diagrammed consistently.

Vision Transformer

```
N = (H/P_h)*(W/P_w)
patch_vector = P_h*P_w*C
tokens_with_CLS = N+1
patch_projection = (patch_vector+1)*D_e
learned_positions = (N+1)*D_e
CLS_parameters = D_e
```

H, W: image size; P_h, P_w: patch size; C: channels; N: patches; D_e: embedding dimension.

Old-paper anchor: 256 x 256 x 3, patch 32 -> 64 patches, vector 3072, 65 tokens. With D_e=128, projection 393344, positions 8320.

Autoencoder and GAN

```
z = Encoder(x)
x_hat = Decoder(z)
MSE = (1/n)sum_i(x_i-x_hat_i)^2
```

z: latent code; x_hat: reconstruction; n: number of values.

```
min_G max_D E_x[ln D(x)] + E_z[ln(1-D(G(z)))]
non_saturating_generator_loss = -E_z[ln D(G(z))]
```

$G(z)$: generated sample from noise z ; $D(x)$: real probability; E : expectation/average.

Training cue: real+fake $\rightarrow$ update D ; fresh noise $\rightarrow$ update G through D ; alternate.

Pix2Pix vs CycleGAN

Pix2Pix	CycleGAN
Paired aligned data	Unpaired domain sets
U-Net generator, PatchGAN	Two residual generators, two discriminators
Adversarial + paired L1	Adversarial + cycle consistency

```
L_cyc = E_x[||F(G(x))-x||_1] + E_y[||G(F(y))-y||_1]
```

Non-overlapping 256/64 patches: $(256/64)^2=16$. Sliding PatchGAN needs stride and padding.

Seven old-case convolutions $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 2048$, all 3×3 with bias: 25, 164, 608 parameters for those stated layers only.

Adversarial attacks

```
x_adv = clip(x + epsilon*sign(gradient_x L(theta,x,y)), valid_range)
```

ϵ : maximum step/budget; $\text{gradient}_x L$: loss slope with respect to input; sign : component-wise direction; clip : enforce pixel and perturbation range.

FGSM is one step. BIM repeats step size α and projects into the allowed ϵ neighborhood.

Poisoning attacks training; evasion attacks inference. Targeted chooses a desired wrong class; untargeted causes any error. White-box knows internals; black-box does not.

Regularization

```
L_L1 = L_data + lambda*sum_i abs(theta_i)
L_L2 = L_data + lambda*sum_i theta_i^2
```

λ : regularization strength. L1 promotes sparsity/zeros; L2 smooth shrinkage. Dropout masks training activations; augmentation adds valid variations; early stopping monitors validation loss.

Architecture one-liners

Model	Must-write phrase
VGG	repeated 3×3 ; depth and extra nonlinearities
Inception	parallel scales, concatenate, 1×1 bottlenecks
ResNet	$H(x)=F(x)+x$; direct gradient path
U-Net	encoder-decoder with matching skip concatenations
Dilated CNN	larger field without more kernel weights
ViT	patches $\rightarrow$ embeddings+positions $\rightarrow$ Transformer $\rightarrow$ CLS
LSTM	cell state plus forget/input/output gates
GRU	reset/update gates, fewer parameters than LSTM
Autoencoder	reconstruct through latent bottleneck
GAN	generator-discriminator adversarial game

High-yield distinctions

A	B
ResNet skip adds	U-Net skip concatenates
Semantic: class per pixel	Instance: separate objects
L1: sparse	L2: shrink
Pix2Pix: paired	CycleGAN: unpaired
Poisoning: training	Evasion: inference
Transfer: reuse knowledge	Domain adaptation: distribution shift
LIME: local surrogate	SHAP: contribution allocation
Feature extraction: freeze	Fine-tuning: update pretrained layers
Key: matched	Value: retrieved
Validation: tune/stop	Test: final estimate

Last check before you enter

- Write assumptions before an ambiguous numerical.
- Never omit input channels from a convolution parameter count.
- Never multiply recurrent parameters by sequence length.
- Pooling has no trainable parameters.
- A ViT CLS token adds one token; state whether you included it.
- A correct-class probability near 1 must give cross-entropy near 0.
- Clip FGSM pixels after perturbing.
- For a five-mark theory answer: definition, flow, purpose, example, limitation.
- Draw labeled architecture diagrams when possible.

Calm is a scoring strategy: write the formula first, then let the formula carry the arithmetic.